

BLE Security with STM32WB



Agenda Theory

1 Introduction to BLE security

2 Introduction to Security
related API of BLE Stack

Agenda

Hands-on and Demo

3 Simple BLE app generated from STM32CubeMX

4 Pairing, Just works

5 Pairing, Passkey entry Display only

6 Pairing, Passkey entry Keyboard only

7 Pairing Numeric comparison

8 Bonding

9 Demo Pairing via NFC

10 Authorization

Thank you

Introduction to BLE security

Security of Bluetooth Low Energy protocol

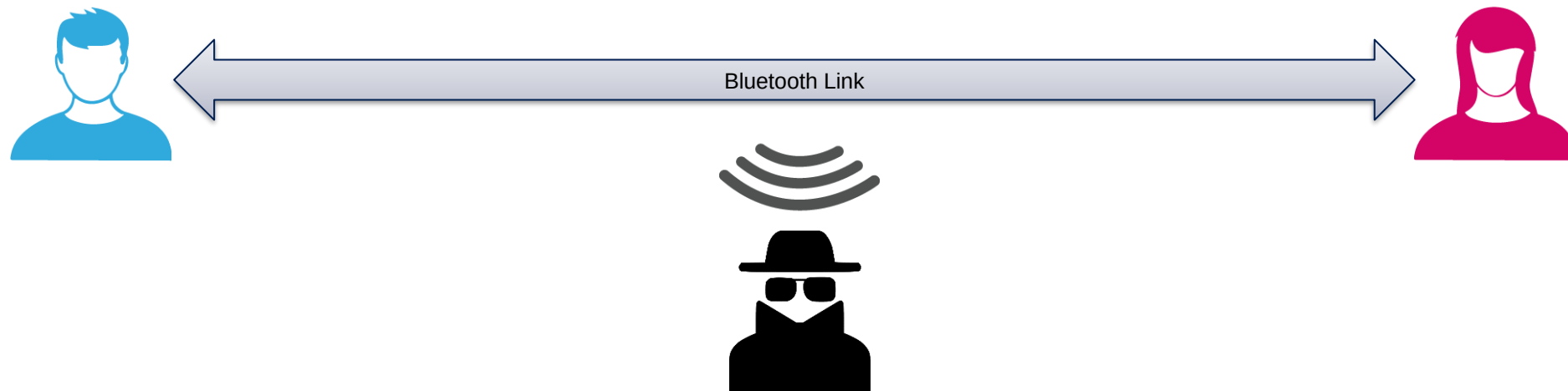
- Confidentiality of data
- Integrity of data
- Authenticity of both devices creating the link
- Authorization – restriction of access to data stored inside the device



Bluetooth Security Threats

Passive Eavesdropping

Passive eavesdropping is where a third party listens in to the data being exchanged between two paired devices.



Bluetooth Security Threats

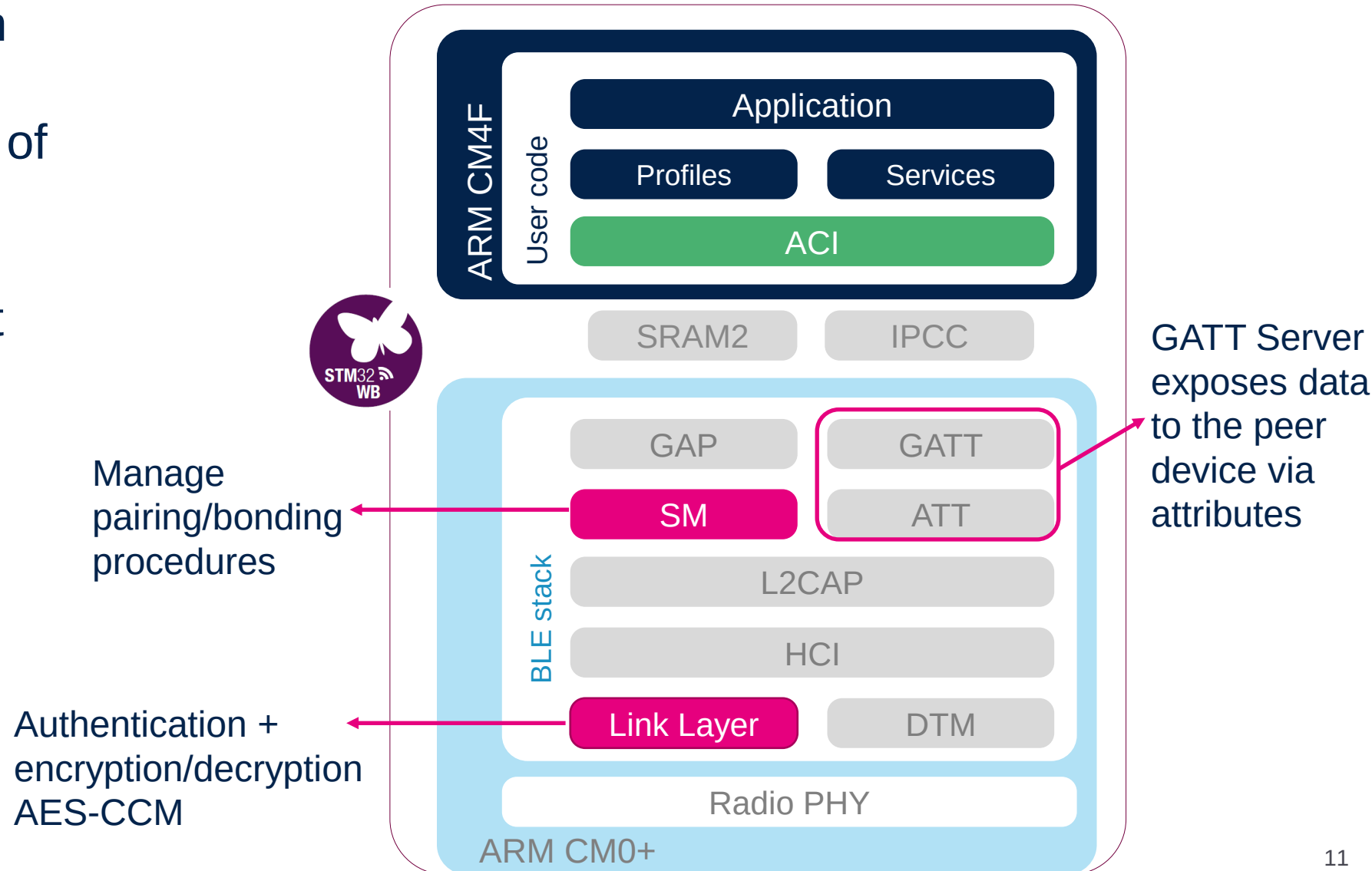
Man In The Middle

MITM (i.e. active eavesdropping) is a process by which a third party impersonates the Initiator and Responder, in order to trick them into connecting to it. The BLE central and peripheral devices will connect to the malicious party which intercepts all data being sent between them. The malicious party can also insert false data or remove data before it reaches the recipient.



BLE Stack architecture on STM32WB

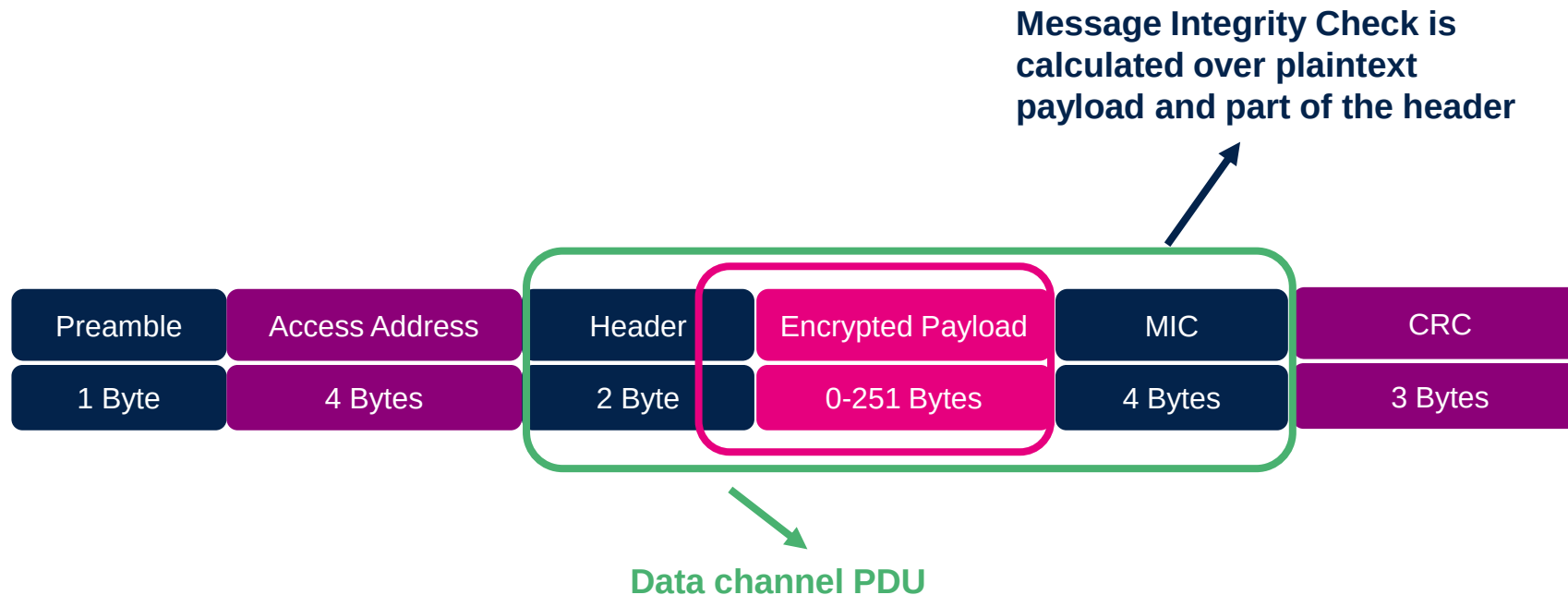
- Application running on CM4 core configures security parameters of BLE Stack via ACI commands and responds to ACI event
- GATT server exposes data to peer device (Client) via attributes
- BLE Service may require that data exchange is secured and protected



Link layer security

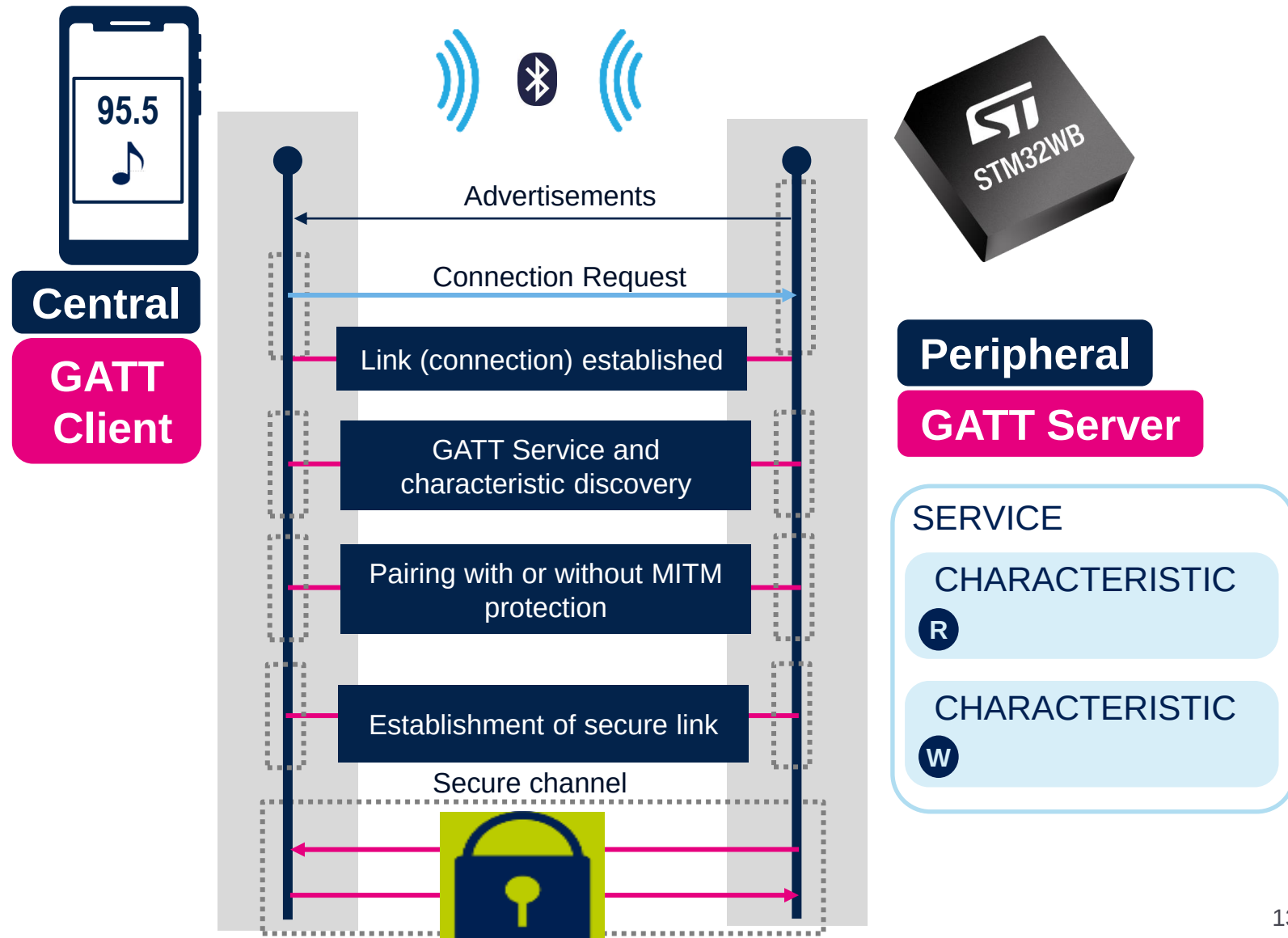
Encrypted link

- After secure link establishment, every data packet is protected by AES-CCM (AES Counter mode with CBC-MAC)



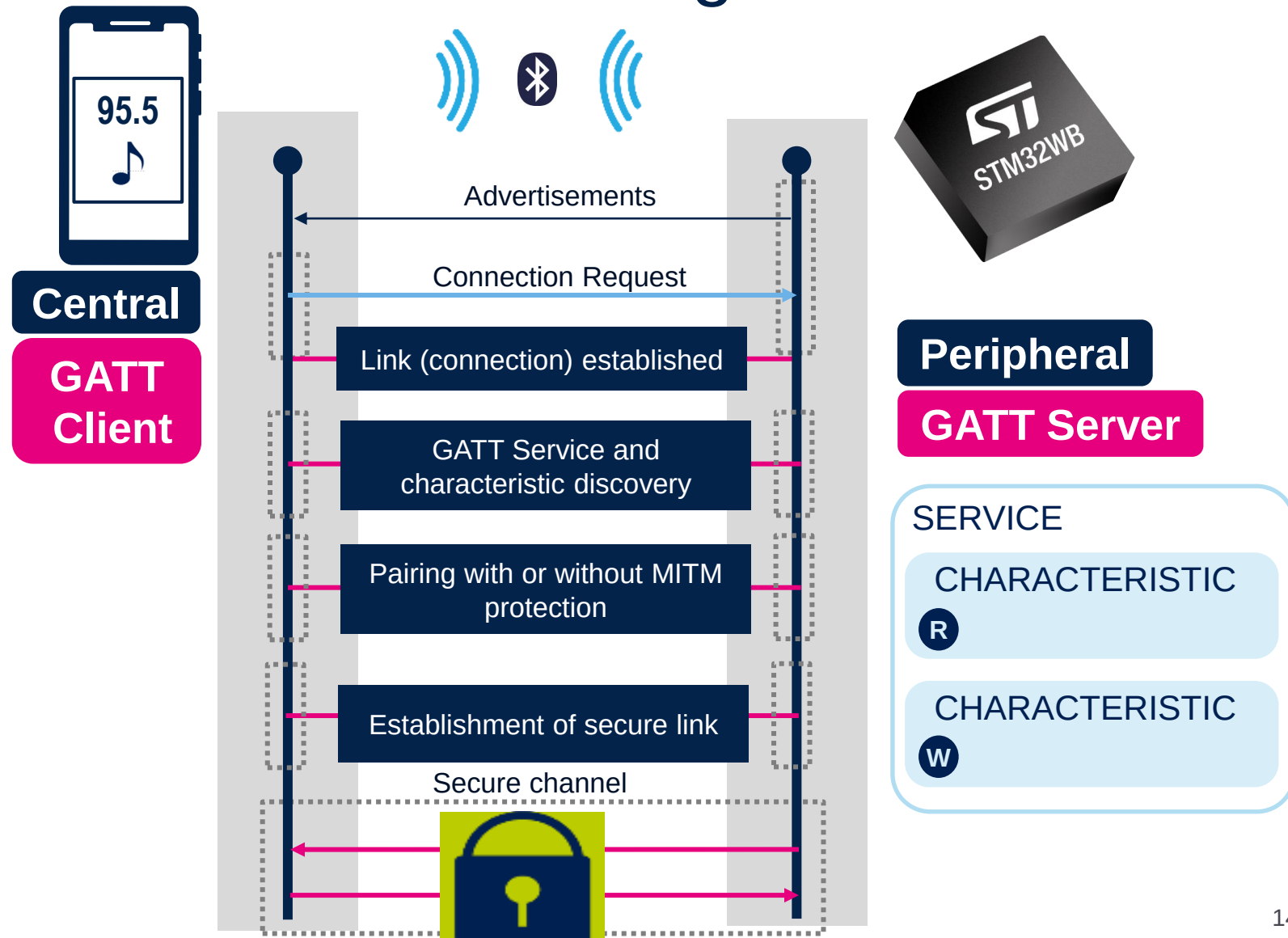
Link layer security AES-CCM

- Peripheral device broadcasts advertising packets
- Central device initiates connection
- GATT Client performs discovery of services and characteristics
- Pairing: creating shared secret keys
- Link is encrypted with a key derived from a shared secret



Pairing Crating shared secrets

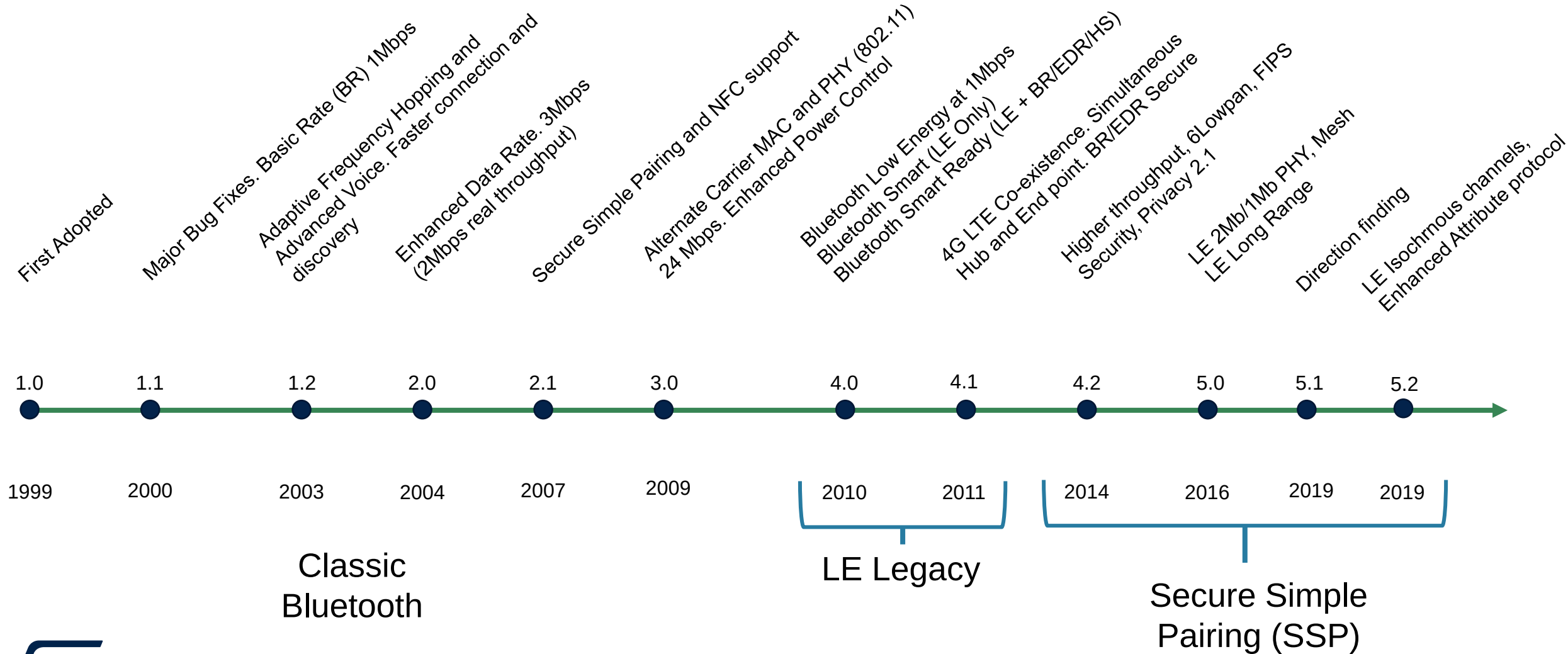
- Level of security depends on type of pairing method which can provide protection against
 - Eavesdropping
 - Eavesdropping + Man In The Middle (Authenticated link)
 - Neither i.e. Pairing must be done in secure environment only. Subsequent connection are secure even if established in non-secure environment



Keys generated during pairing

- LTK (Long term key), STK (Short term key)
Used for confidentiality and message authenticity on the Link layer. Device authentication depends on pairing method used. Not all methods provide device authentication!
- IRK (Identity Resolving Key)
Used to provide protection against Identity tracking. Privacy feature
- CSRK (Connection Signature Resolving Key)
Authentication of unencrypted data on ATT layer

Bluetooth Evolution



Association models (Pairing methods)

- LE Legacy
 - Just Works (No user interaction required)
 - Passkey Entry
 - Out of Band (typically over NFC)
- Secure Simple Pairing (SSP)
 - Just Works
 - Passkey Entry
 - Numeric Comparison
 - Out of Band
- LE Legacy pairing doesn't provide protection against eavesdropping. Pairing must be done in secure environment! The exception is Out of Band which is secure provided that the OoB channel is secure
- For Secure simple pairing, all methods provide protection against eavesdropping thanks to ECDH (Elliptic-curve Diffie–Hellman key agreement)
- Just works provide no protection against MITM. There is no device authentication!

Pairing via Just Works

1. First, power on the device
2. Scan for devices to connect
3. Select device to pair to
4. Devices are paired



Pairing via Passkey entry

1. First, power on the device
2. Scan for devices to connect
3. Select device to pair to
4. Confirm PIN (6 decimal digits)
5. Devices are paired



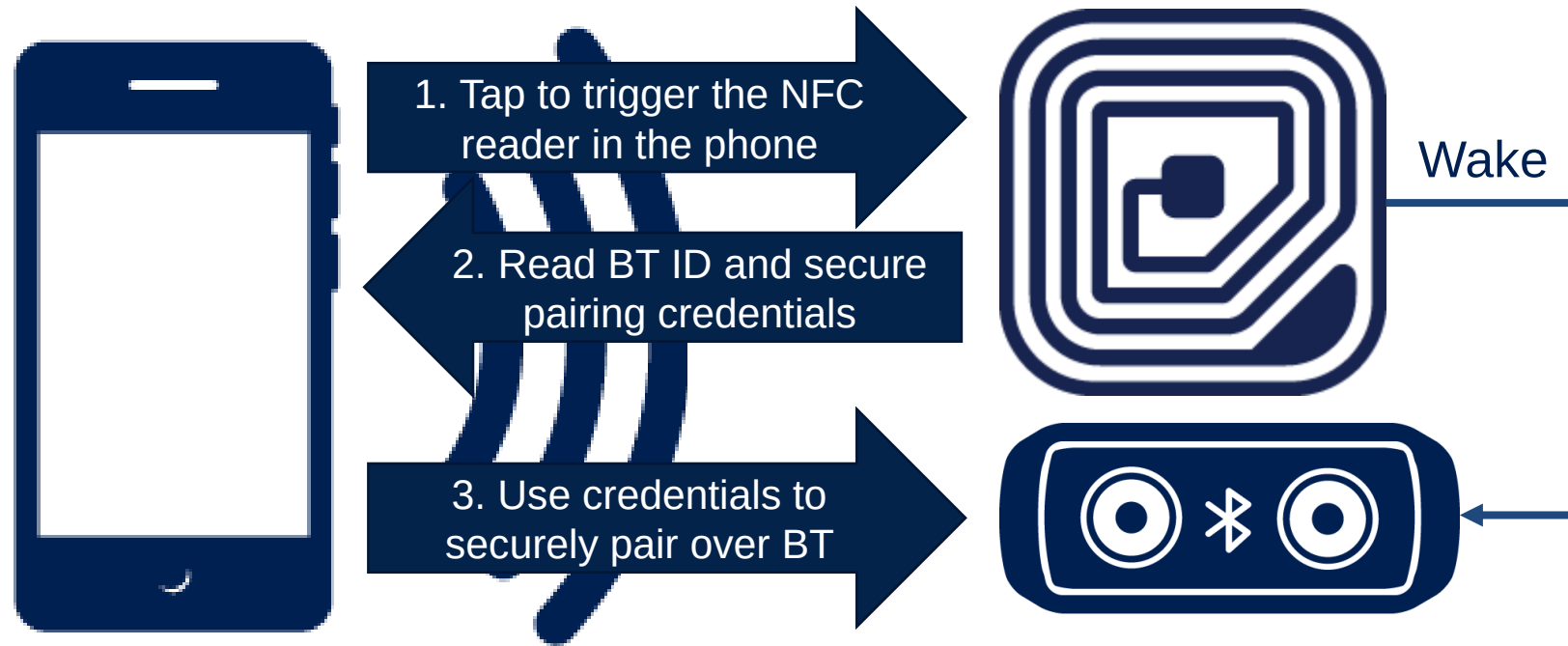
Paring via Numeric Comparison (only SSP)

1. First, power on the device
2. Scan for devices to connect
3. Select device to pair to
4. Confirm PIN (6 decimal digits)
5. Devices are paired



Pairing via Out of Band (NFC use case)

- NFC simplifies the pairing process



Benefits of NFC pairing to end-customer

- **Faster and simpler** – no need for BT/Wi-Fi sub-menus or searching through lists to find surrounding devices
- **No conflicts** – pair only the devices you intend to pair
- **Secure communications** – encrypt a link by exchanging credentials securely with just a tap
- **Low cost** - ideal for BT devices that are too small or cannot afford a user interface for PIN or password entry
- **Ease of use** - unlike QR codes, Line of Sight is not required with NFC
- **Save power** – use NFC to automatically turn on a battery-powered BT device and instantly pair



Device options for pairing

ST25T Tags

Basic NFC pairing responder solution
Ideal for simple and low-cost applications



ST25D Dynamic Tags

Secure NFC pairing responder solution
NFC tag with I²C. Ideal for embedded systems



ST25R Readers

Secure NFC pairing initiator and responder
Highly flexible. Perfect for dual role devices



Thank you

Introduction to Security related API of BLE Stack

What will we learn?

How to select the pairing method in User application?

How to initiate pairing?

References

- **AN5270 STM32WBx5 Bluetooth® Low Energy (BLE) wireless interface**
- PM0271 STM32WB BLE stack programming guidelines
- AN5289 Building wireless applications with STM32WB Series microcontrollers
- STM32 MOOCs (Massive Open Online Courses)
- Bluetooth Core Specification

Association models (Pairing methods)

- Prior to pairing devices execute Pairing feature exchange
 - IO capabilities
 - Whether or not protection against MITM is required
 - Whether OOB data have been received previously (ex. via NFC)
 - Support of SSP (Secure simple pairing)
- Pairing method selection depend on all above

Initiator/ Responder	Display only	Display and Yes/No input	Keyboard only	No input/output	Keyboard and display
Display only	Just works	Just works	Passkey Entry	Just works	Passkey Entry
Display and Yes/No input	-	Just works (Legacy)	Passkey Entry	Just works	Passkey Entry (Legacy)
		Numeric comparison (SSP)			Numeric comparison (SSP)
Keyboard only	-	-	Passkey Entry	Just works	Passkey Entry
No input/output	-	-	-	Just works	Just works
Keyboard and display	-	-	-	-	Passkey Entry (Legacy)
					Numeric comparison (SSP)

Pairing feature exchange IO capabilities

```
//      - 0x00: IO_CAP_DISPLAY_ONLY
//      - 0x01: IO_CAP_DISPLAY_YES_NO
//      - 0x02: IO_CAP_KEYBOARD_ONLY
//      - 0x03: IO_CAP_NO_INPUT_NO_OUTPUT
//      - 0x04: IO_CAP_KEYBOARD_DISPLAY
tBleStatus aci_gap_set_io_capability( uint8_t IO_Capability )
```

This function should be called during GAP/GATT initialization phase.

Pairing feature exchange MITM and OOB

- Apart from IO capabilities, BLE devices exchange also
 - OOB flag (Out of band data from the other device was received)
 - MITM flag (protection against Man in the Middle is required)

Initiator/ Responder	OOB Set	OOB Not Set	MITM Set	MITM Not Set
OOB Set	Use OOB	Use OOB		
OOB Not Set	Use OOB	Check MITM		
MITM Set			Use IO capabilities	Use IO capabilities
MITM Not Set			Use IO capabilities	Use Just Works

Applies only to SSP. For Legacy pairing check BLE specification

Setting MITM flag

```
//          - 0x00: MITM protection not required
//          - 0x01: MITM protection required

tBleStatus aci_gap_set_authentication_requirement( uint8_t Bonding_Mode,
                                                    uint8_t MITM_Mode,
                                                    uint8_t SC_Support,
                                                    uint8_t KeyPress_Notification_Support,
                                                    uint8_t Min_Encryption_Key_Size,
                                                    uint8_t Max_Encryption_Key_Size,
                                                    uint8_t Use_Fixed_Pin,
                                                    uint32_t Fixed_Pin,
                                                    uint8_t Identity_Address_Type )
```

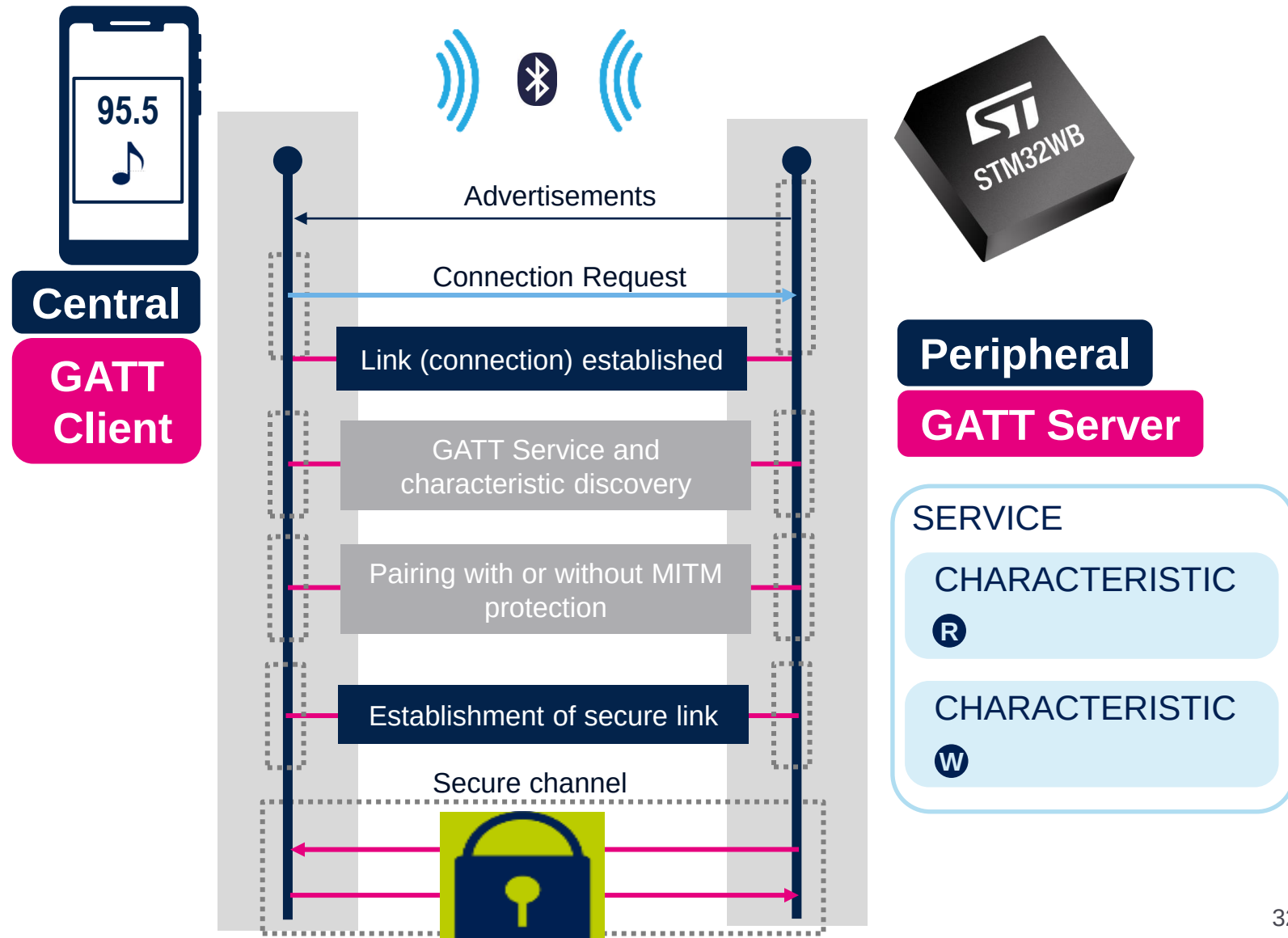
Secure simple pairing vs Legacy

```
//      - 0x00: Secure Connections Pairing not supported
//      - 0x01: Secure Connections Pairing supported but optional
//      - 0x02: Secure Connections Pairing supported and mandatory (SC Only Mode)

tBleStatus aci_gap_set_authentication_requirement( uint8_t Bonding_Mode,
                                                    uint8_t MITM_Mode,
                                                    uint8_t SC_Support,
                                                    uint8_t KeyPress_Notification_Support,
                                                    uint8_t Min_Encryption_Key_Size,
                                                    uint8_t Max_Encryption_Key_Size,
                                                    uint8_t Use_Fixed_Pin,
                                                    uint32_t Fixed_Pin,
                                                    uint8_t Identity_Address_Type )
```

Bonding

- Bonding information = security and identity information
- Act of storing bonding information inside security database for use in subsequent connections
- Both BLE devices must be in bonding mode for bonding to take place
- Service discovery takes time and consumes energy. GATT client can cache the information for bonded device

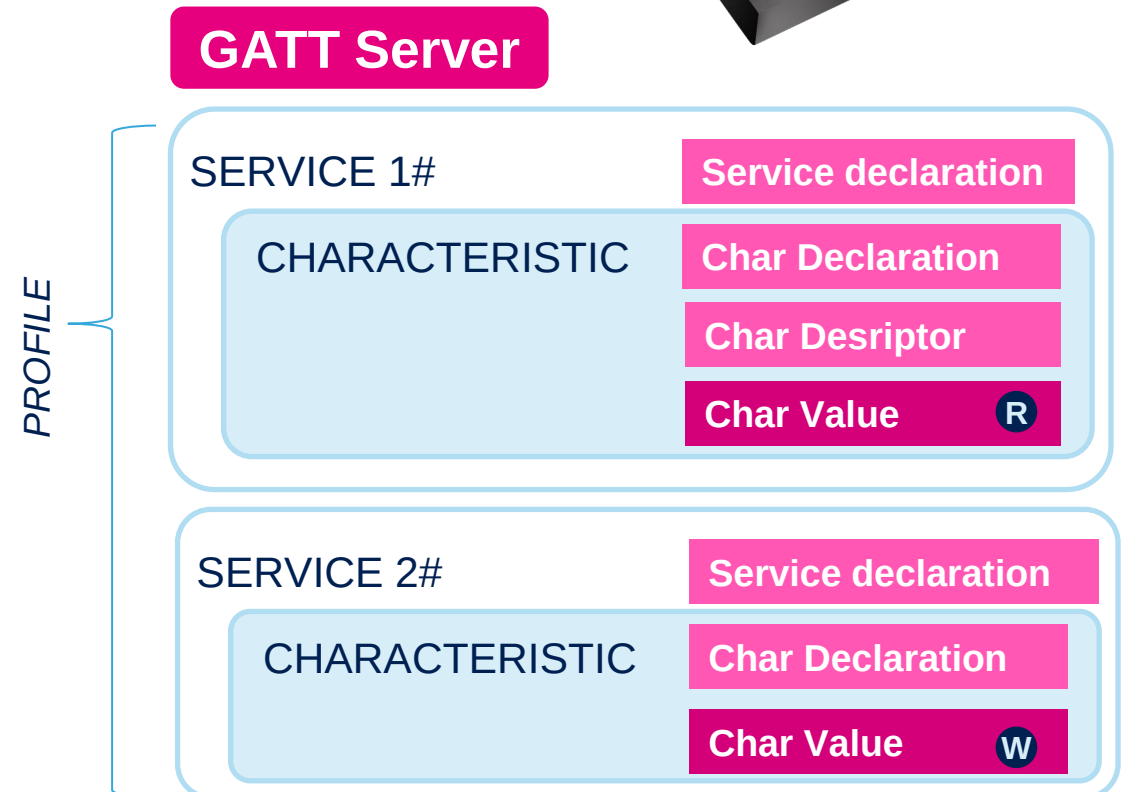


Allow bonding

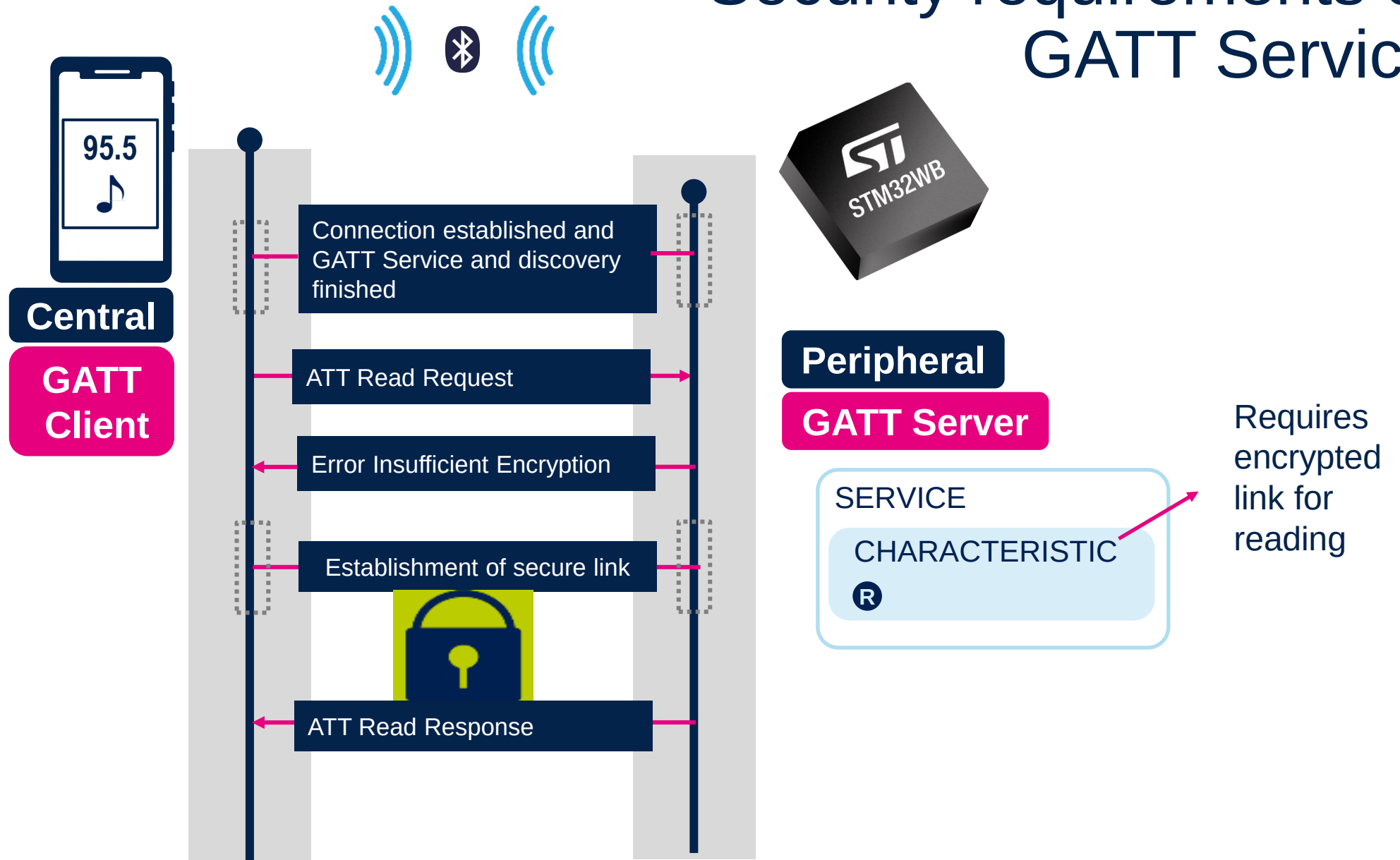
```
//          - 0x00: No-bonding mode
//          - 0x01: Bonding mode
tBleStatus aci_gap_set_authentication_requirement( uint8_t Bonding_Mode,
                                                    uint8_t MITM_Mode,
                                                    uint8_t SC_Support,
                                                    uint8_t KeyPress_Notification_Support,
                                                    uint8_t Min_Encryption_Key_Size,
                                                    uint8_t Max_Encryption_Key_Size,
                                                    uint8_t Use_Fixed_Pin,
                                                    uint32_t Fixed_Pin,
                                                    uint8_t Identity_Address_Type )
```

Security requirements of GATT Service

- Characteristic may require establishment of Secure link before the **value** can be read or written by GATT Client. It may require
 - Encrypted physical link
 - Authenticated (MITM protected) + Encrypted link
- When security requirements are not met, GATT server sends Error response:
 - “Insufficient Encryption
 - “Insufficient Authentication”GATT client may request GAP to secure link with previously shared key or initiate pairing
- List of Services and Characteristics is not considered private or confidential and therefore Service and Characteristic discovery shall always be permitted



Security requirements of GATT Service



Security permission for Characteristic

```
aci_gatt_add_char(uint16_t Service_Handle,  
                 uint8_t Char_UUID_Type,  
                 const Char_UUID_t* Char_UUID,  
                 uint16_t Char_Value_Length,  
                 uint8_t Char_Properties,  
                 uint8_t Security_Permissions,  
                 uint8_t GATT_Evt_Mask,  
                 uint8_t Enc_Key_Size,  
                 uint8_t Is_Variable,  
                 uint16_t* Char_Handle )
```

Security_Permissions

- 0x00: None
- 0x01: AUTHEN_READ (Need authentication + encryption to read)
- 0x04: ENCRY_READ (Need encryption to read)
- 0x08: AUTHEN_WRITE (Need authentication + encryption to write)
- 0x20: ENCRY_WRITE (Need encryption to write)



PROFILE

GATT Server

SERVICE 1#

Service declaration

CHARACTERISTIC

Char Declaration

Char Desriptor

Char Value R N

SERVICE 2#

Service declaration

CHARACTERISTIC

Char Declaration

Char Value R

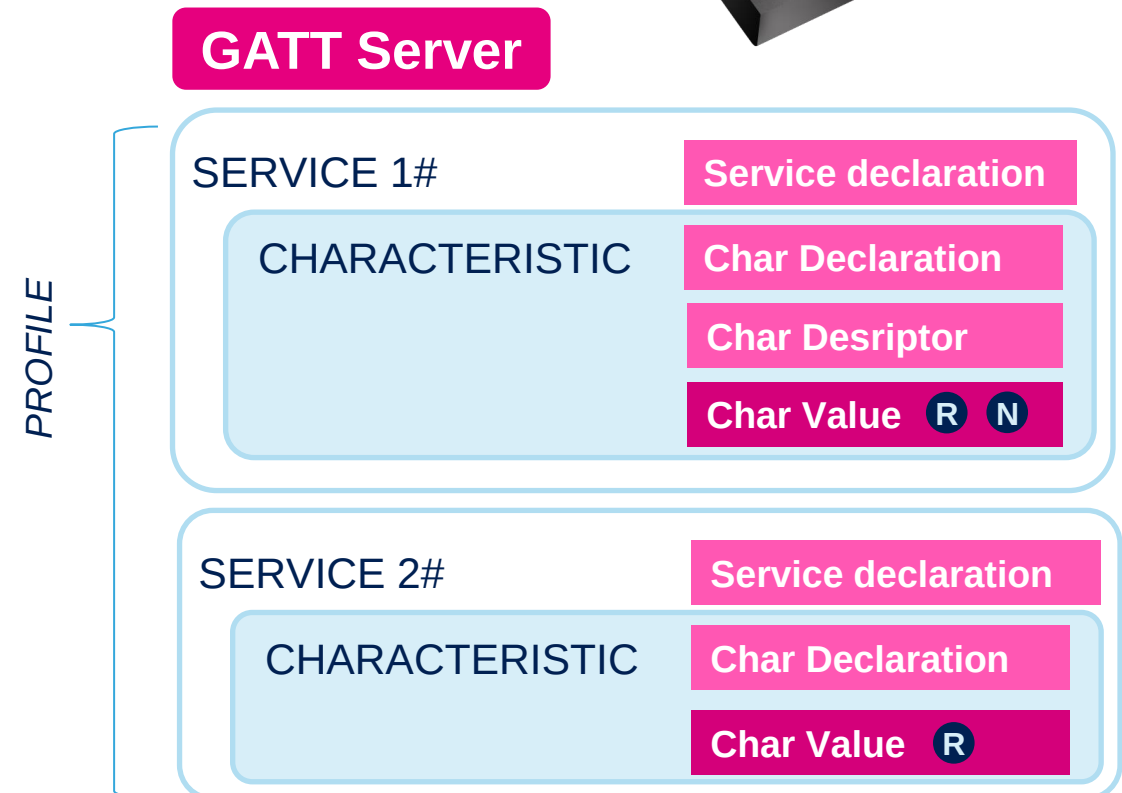
Authorization requirements of GATT Service

- Characteristic may also require authorization i.e. Confirmation from User app that GATT Client can read or write to the characteristic
- Authorization requirements are handled by the Application layer unlike the Security requirements which are handled by BLE Host

```
aci_gatt_add_char(uint16_t Service_Handle,  
                 uint8_t Char_UUID_Type,  
                 const Char_UUID_t* Char_UUID,  
                 uint16_t Char_Value_Length,  
                 uint8_t Char_Properties,  
                 uint8_t Security_Permissions,  
                 uint8_t GATT_Evt_Mask,  
                 uint8_t Enc_Key_Size,  
                 uint8_t Is_Variable,  
                 uint16_t* Char_Handle )
```

Security_Permissions

- 0x02: AUTHOR_READ (Need authorization to read)
- 0x10: AUTHOR_WRITE (Need authorization to write)



Security modes defined by BLE Specification

- Security Mode 1 – Link layer encryption
 - Level 1 – No Security (No authentication & No encryption)
 - Level 2 – Unauthenticated pairing with encryption
 - Level 3 – Authenticated pairing with AES-CCM encryption
 - Level 4 – LE Secure Connections pairing with ECDH & AES-CCM encryption
- Security Mode 2 - Data signing on ATT layer

Shall not be used together with encrypted link i.e. Security Mode 1, Level 2/3/4. It is used by services that require fast connection set up. The Signed Write Command is used to request the server to write the value of an attribute with an authentication signature, typically into a control-point attribute. The two devices must be bonded to have the CSRK

 - Level 1 - Unauthenticated pairing with data signing
 - Level 2 – Authenticated pairing with data signing



Peripheral

GATT Server

SERVICE

CHARACTERISTIC

R

CHARACTERISTIC

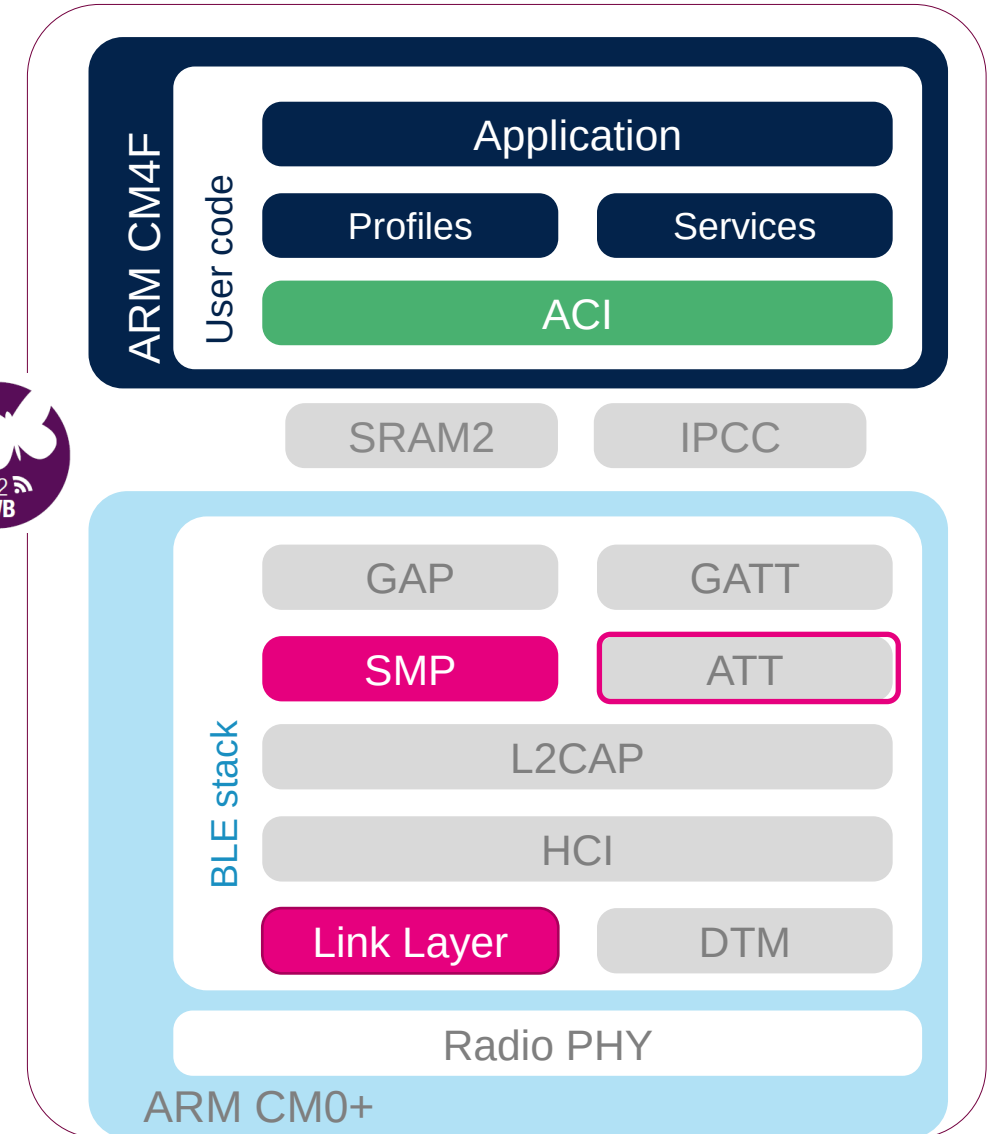
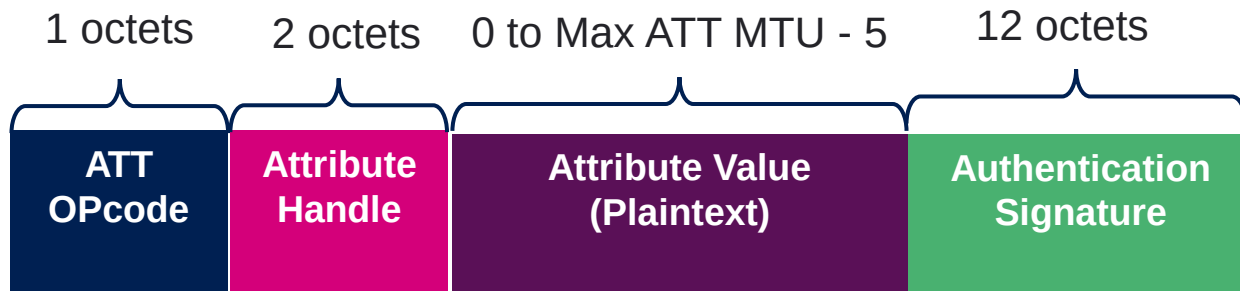
W

ATT layer security

Nonencrypted link with Signed data

- Only for ATT Signed write command
- Both devices must have CSRK from previous pairing/bonding
- Link must be non-encrypted

ATT Signed write command



Security Mode 2 - Data signing on ATT layer

API called by BLE Client

```
tBleStatus aci_gatt_signed_write_without_resp( uint16_t Connection_Handle,  
                                                uint16_t Attr_Handle,  
                                                uint8_t Attribute_Val_Length,  
                                                const uint8_t* Attribute_Val );
```

Thank you